

目录

CONTENTS

- 01 / 零信任代理
- 02 / 领域知识注入
- 03 / Skill 铸造与演化
- 04 / 愿景与展望

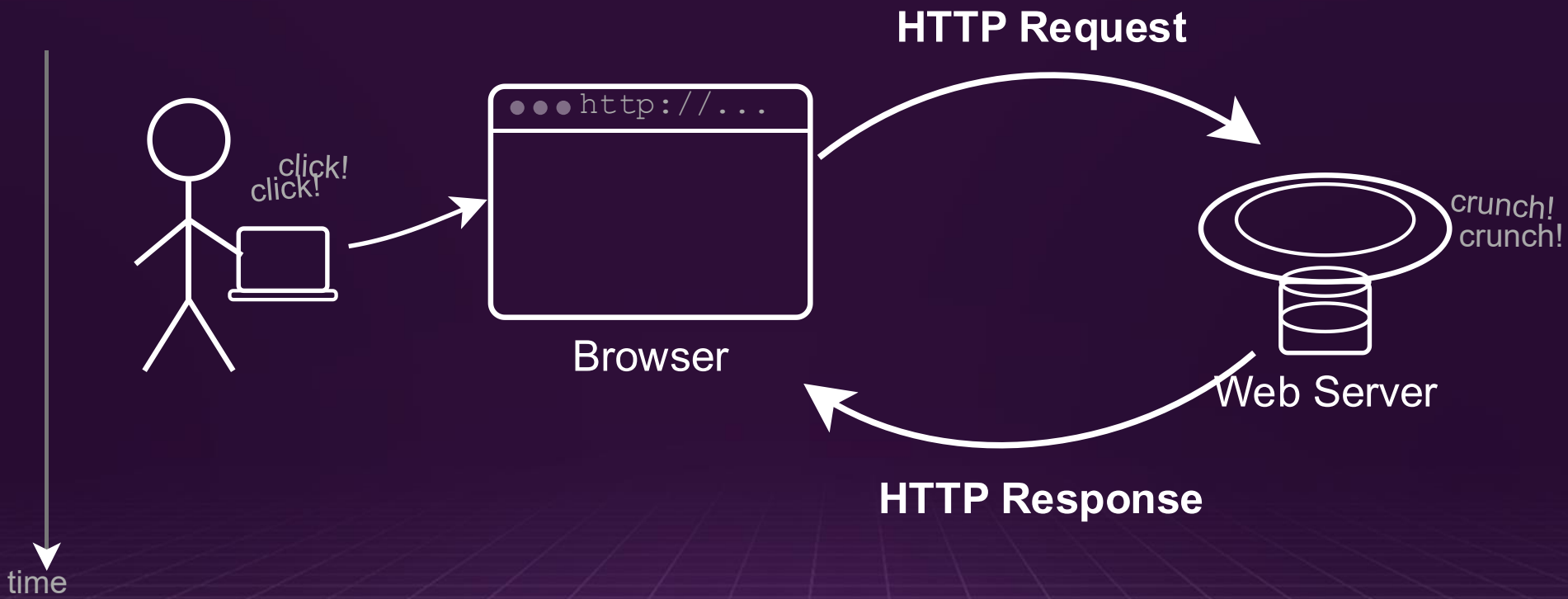
PART ONE

零信任代理

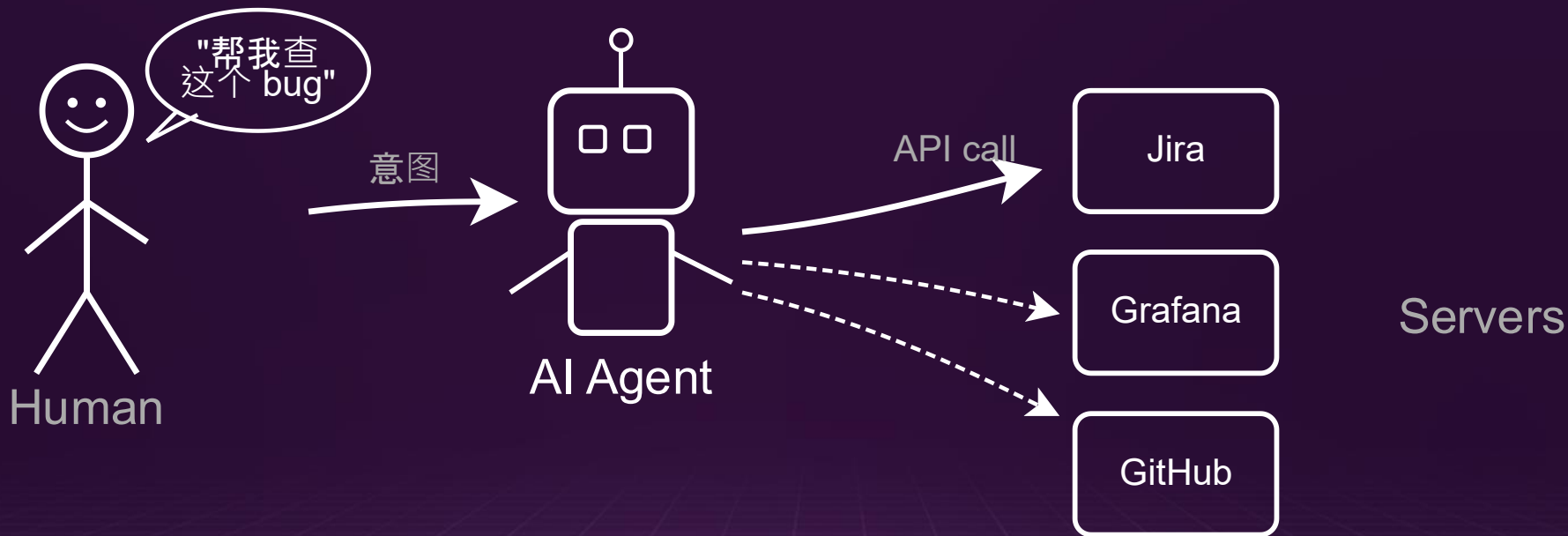
让 AI Agent 进门

01

过去， 人类与系统的交互
是什么样的？



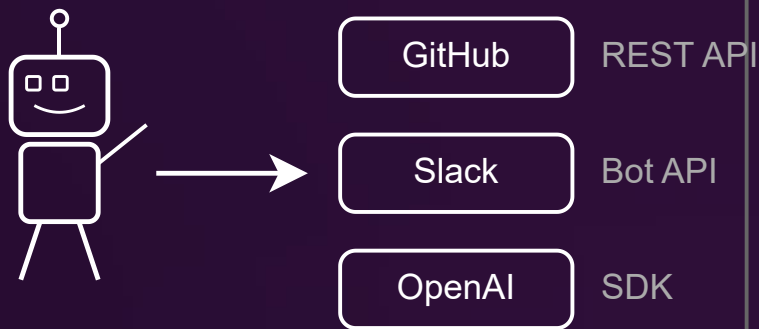
**未来，人类与系统的
交互会变成什么样？**



Agent-Server — 人只说意图, Agent 自动调 API

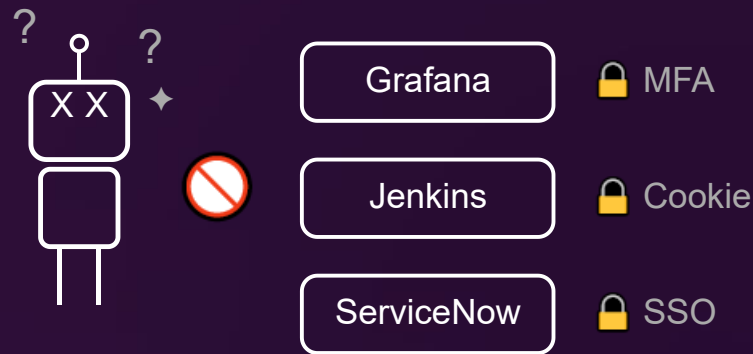
此刻，我们处于一种**过渡阶段**
并不是所有的系统都有 Agent 友好的接口

Agent 友好 ✓

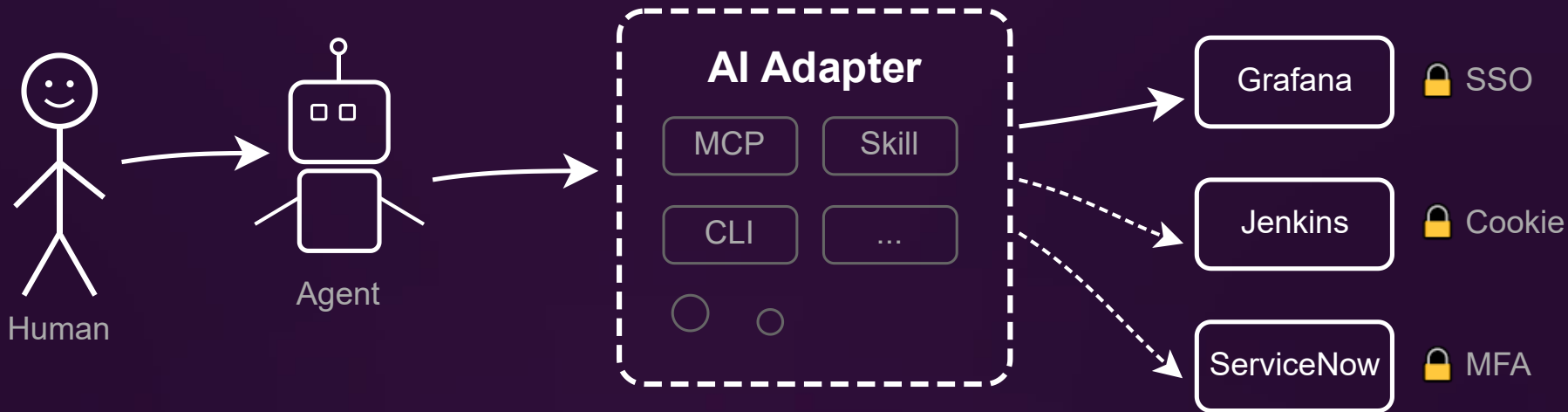


直接调用，畅通无阻

Agent 不友好 ✗

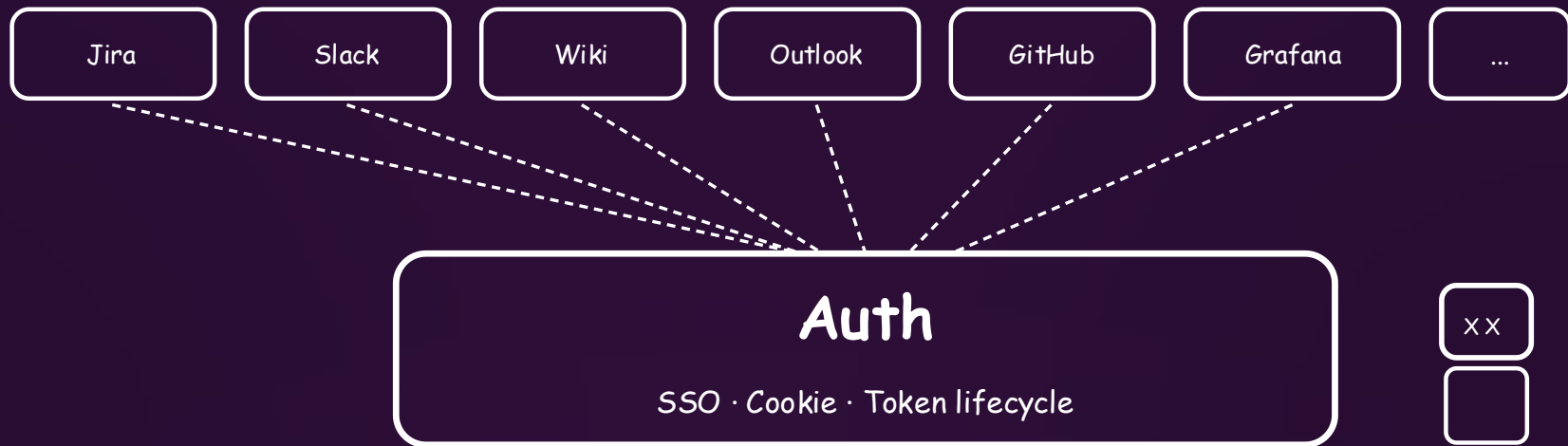


撞墙！需要 AI Adapter



Hybrid — 现实世界需要 AI Adapter

我们连通了 20+ 工作系统
它们的公共挑战在哪里？



公共痛点 = Auth

业界怎么做？

申请 tech token



多数系统不支持

支持的也要走审批

抓 cookie

Name	Value
session_id	a8f3...c91
csrf_token	9bd2...4a7
auth_token	eyJ0eX...
_ga	GA1.2...

翻 DevTools 复制



几小时过期

这些方式有什么安全问题？



Agent

Context Window

[system] You are a helpful assistant...

[user] 帮我查一下 JIRA-1234 的状态

[assistant] 好的, 让我查一下...

[tool_use] headers:

Cookie: JSESSIONID=abc123xyz789secret!

Authorization: Bearer eyJhbGciOi...超长token



明文!

[tool_result] {"status": "In Progress"...}

[assistant] JIRA-1234 当前状态是...

日志

被记录

训练数据

可能泄露

其他工具

可见

Credential 一旦进入 Context Window = 全面暴露

SigCLI

Auth 的公共抽象


```
$ sig login https://jira.example.com
```

```
⋮ launching browser...
```

```
✓ login successful → credentials encrypted & stored
```



sig get

获取 credential headers
用于自己的 HTTP 调用

sig request

直接发起认证请求
像 curl, 但自动带 credentials

sig run

注入环境变量到任意命令
子进程自动获得 credentials

sig proxy

透明 MITM 代理
应用零改动, proxy 自动注入



Agent

Context Window ✓

[system] You are a helpful assistant...

[user] 帮我查一下 JIRA-1234 的状态

[assistant] 好的, 让我查一下...

[tool_use] sig request jira /api/v1/issue/123

// 无 credential, proxy 透明注入

[tool_result] {"status": "In Progress"...

[assistant] JIRA-1234 当前状态是进行中

零 credential 暴露

透明

sig proxy

- ① 截获请求
- ② 解密 credential
- ③ 注入 headers
- ④ 转发到目标



Agent 只看到"发请求 → 得结果", credential 在另一个世界

L1: Platform Connectors

纯 REST 封装 · 无业务逻辑 · 任何项目可复用

Jira

Wiki

Slack

Teams

Outlook

GitHub

Grafana

Jenkins

ServiceNow

Dynatrace

Loki

...

L0: Infrastructure

sig
(SigCLI)

gh

git

node

python3

curl

PART TWO

领域知识注入

知其然，知其所以然

02

AI 到底能多大程度 自动化软件工程师的日常工作?

Anthropic

85%

code is agent-written

Erik Schluntz: 22K line production PR

Cursor / OpenAI

90%+

new code by agent

Greenfield · own codebase

Data source: https://www.youtube.com/watch?v=fHWFF_pnqDk

我们的团队

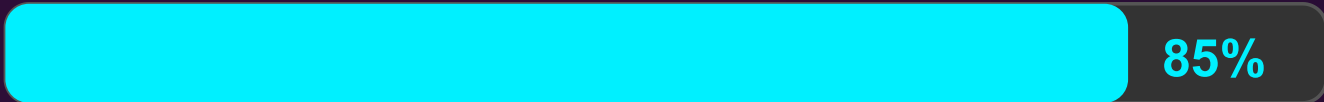
~50%

自动化程度

企业 to-B · 10年代码 · 强领域知识

我们团队的挑战在哪里？

Anthropic



85%

Cursor / OpenAI



90%+

我们团队



~50%

gap

gap = 领域复杂性

排查一个数据状态不一致的问题



"DP 状态异常，我要手动排查..."

先查哪里？不确定...

Source System

打开 → 找 API → 查状态

UCL 同步层

查 assignment → 比对 ID

Catalog 展示层

查 resource → 检查 JOIN

不对，回去再查...

6+ 步 · 3 个系统 · 来回切换

半天 → 还可能查错方向 🕒



"Data Product 一直显示 activating"

"但我确定它已经 active 了啊?"

Source System

Data Product: active ✓

lifecycle = "active"

API resource = enabled

sync

UCL 同步层

ORD resource synced ✓

但 assignment ID 变了...

⚠ stale reference

sync

Catalog 展示层

显示: "activating" ✗

API resource: disabled=true

→ cockpit INNER JOIN 失败

Agent 需要知道 :

查哪层? (不能盲目 resync)

→ 需要逐层排查的诊断逻辑

调哪个 API? (不能猜)

→ 需要精确的 support routes

怎么判断? (业务规则)

→ disabled=true 导致 JOIN 失败

这不是 "写代码" 的问题 — 是 "理解系统" 的问题

我们团队需要领域抽象层

L2

diagnose-product-status

Domain Workflow

← 知道 先查哪层、怎么判断

L1.5

check-sync-layer

查同步状态 + 比对

query-database

连接 + 执行 SQL

inspect-api-resource

检查资源状态

← 知道
怎么操作每一步

L1

grafana-connector

database-connector

service-connector

← 知道
连哪里、怎么认证

L0

sig

curl

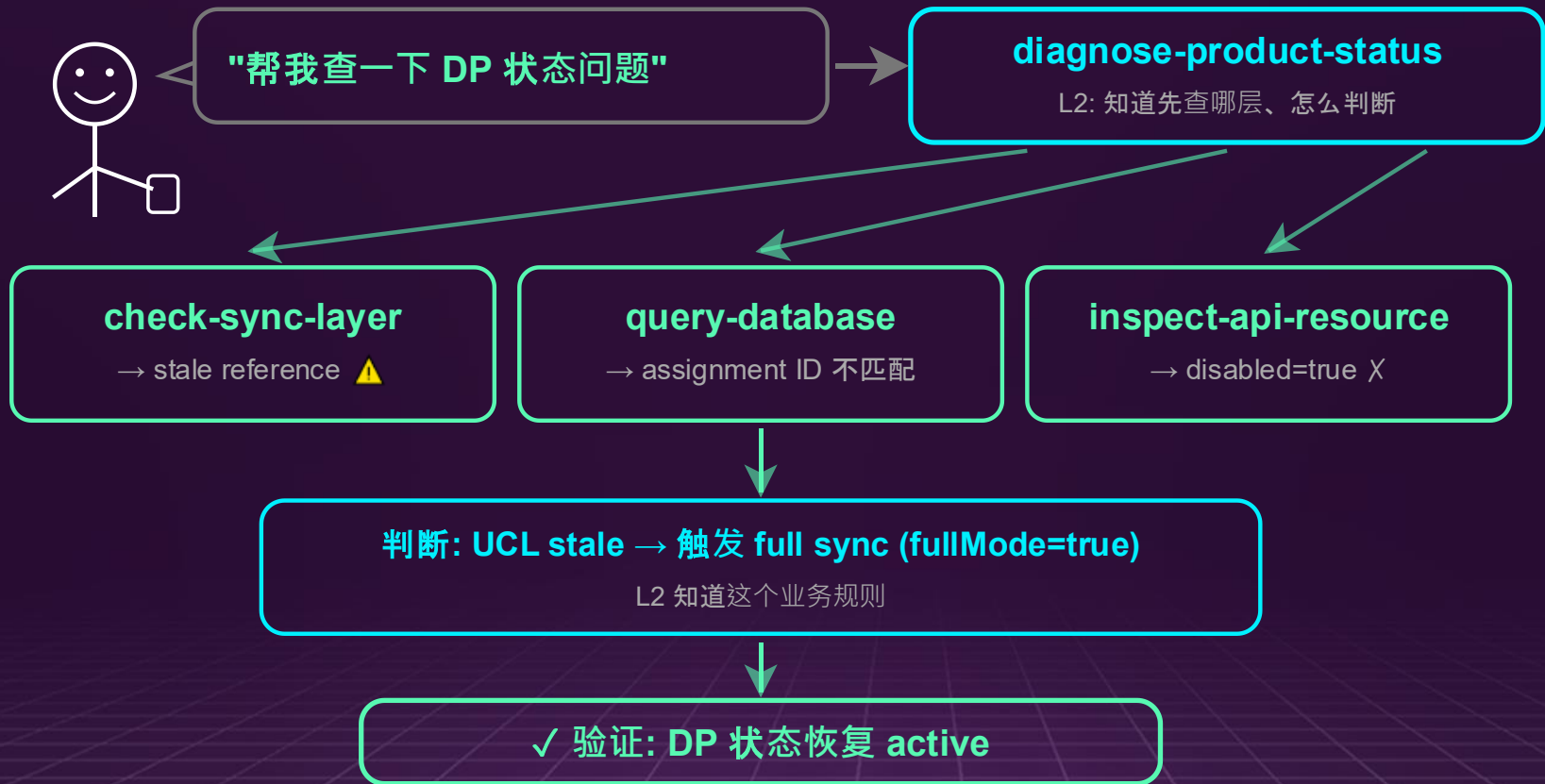
node

python

sig

curl

← 知道
怎么执行



人说一句话 → Skill 分层自动执行 → 几分钟搞定

L2: Domain Workflow

业务知识 + 判断逻辑 · 每人贡献一个领域

diagnose-status

create-pr

debug-package

write-procedure

...

L1.5: Domain Primitives

原子操作 · 无业务判断 · 跨多个 L2 复用

connect-db

run-sql

get-tenant

query-logs

...

L1: Foundation Skills

纯 REST 封装 · 无业务逻辑 · 任何项目可复用

Jira

Slack

GitHub

Grafana

Outlook

...

L0: Infrastructure

auth + tools — 一切的基础

sig

gh

git

node

python

curl

...

PART THREE

铸造和演化

一劳永逸

03

我们不做重复劳动

第 1 次

DP 状态异常 → 手动逐层查
花了半天



第 2 次

又一个 DP 状态问题
同样的步骤，又半天



第 3 次

又来了！？
等等... 这该自动化了

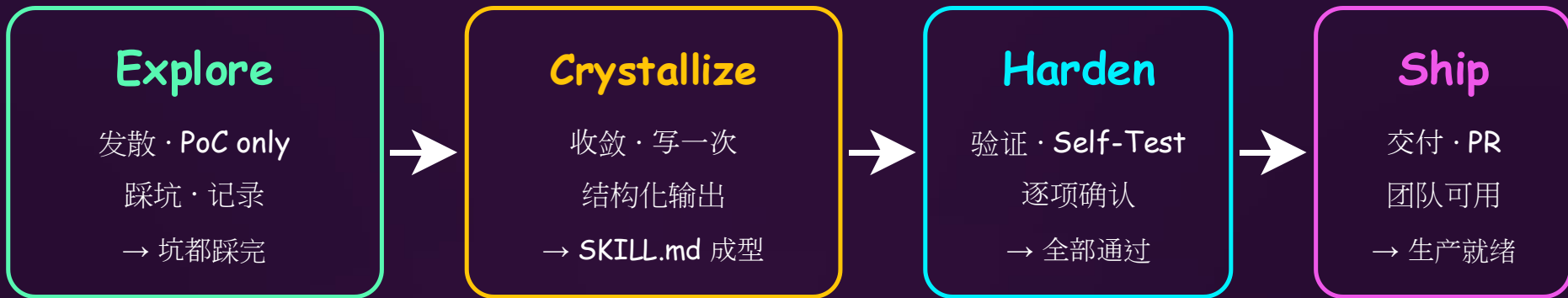
"3x Rule"

同样的事做了 3 遍 → 该变成 Skill 了



→ **Forge Skill**

从散乱经验到可执行 Skill



核心：PoC 和 Skill 写作分离 — 避免 $O(n^2)$ 重写

每个阶段有明确交付物，通过才进入下一阶段



同一 Skill 时好时坏

不同人/环境结果不同

Agent 幻觉走错路

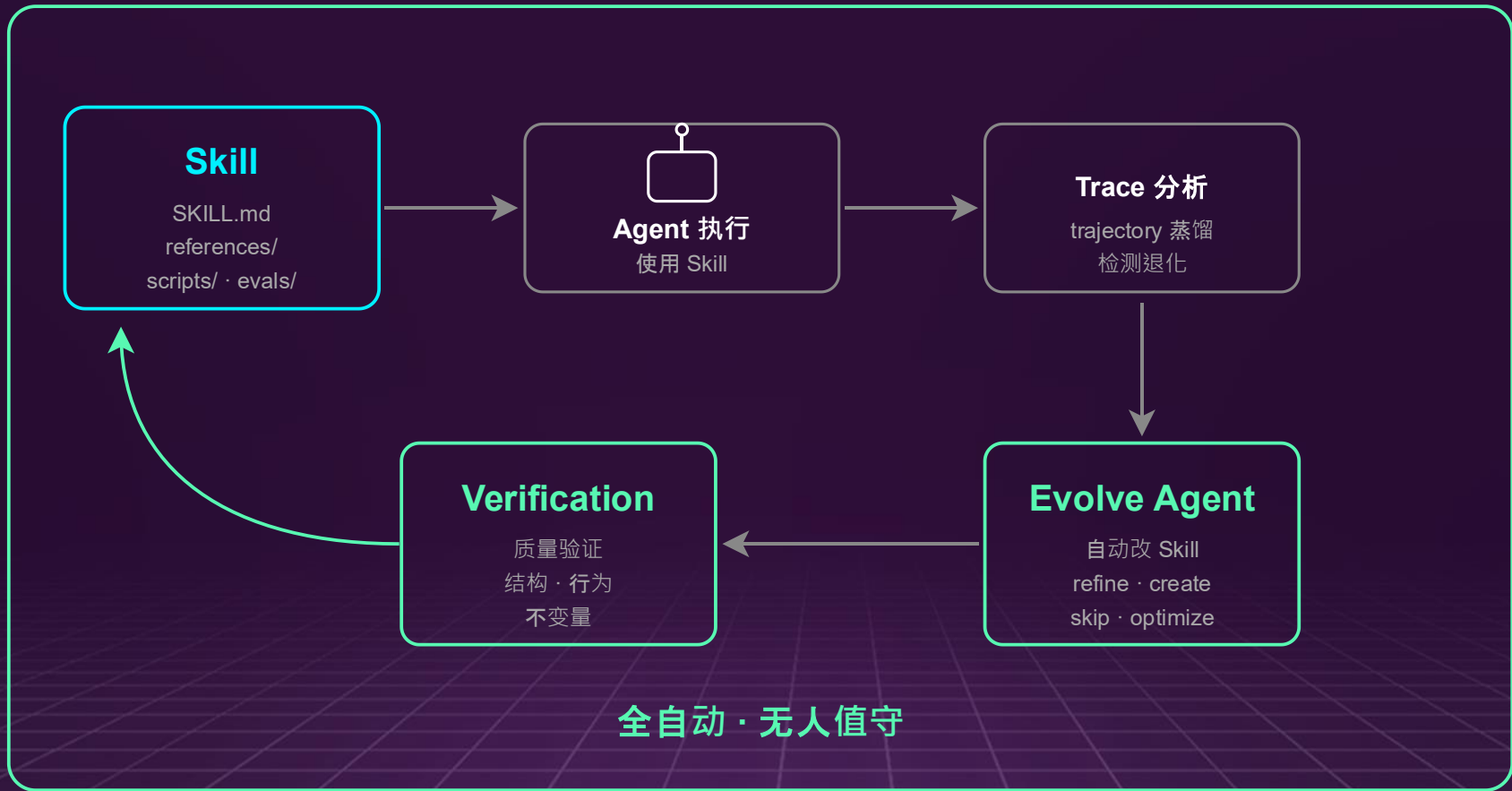
约束不够 → 乱猜

无人维护 → 静默失效

没有 feedback loop

Skill 写完 永远可用

Skill 如何自进化?



如何做 Trace 分析?

Layer 1 — Trajectories (Raw Sessions)

```
{type:"user", message:"https://jira.tools.sap/browse/ DW101-143584 RCA" 309 行  
{type:"assistant", tool_use:" sap-jira 6 类型  
{type:"queue-op", ...} {type:"snapshot", ...} {type:"progress", ...} 700 KB
```

↓ structure · pair action ↔ feedback · drop noise

Layer 2 — Causal Info (per-session)

```
[STEP 2] action: sig request .../DW101-143584 --format body ~12 steps  
feedback: ⚠ Output too large (36 KB) action ↔ feedback  
causal_link: oversize → must narrow query paired · ≤400 chars  
-----  
[STEP 3] action: sig request .../DW101-143584 ?fields=summary,status,priority newdata:  
feedback: ✓ 200 cache-hit (3 KB) _skills_referenced  
causal_link: narrowed-after-oversize _avg_prm
```

↓ aggregate · group by skill across sessions

Layer 3 — Evidence Base (grouped by skill)

G(sap-wiki)

3 sessions · all err

1eb0fca6 · 14 / 95 / 344 steps

→ create

G(sap-jira)

1 session

9e739393 · 95 steps

→ refine

G(sap-loki)

2 sessions · all err

9e739393 · 95 / 344 steps

→ skip

如何验证演化的结果的**有效性**？

```
{
  "id": 1,
  "prompt": "查 DW101-143584 这个 Jira 工单" ,
  "assertions": [

    substring  — 关键字段对了？
    { "type": "substring" , "needle": "sig request" },

    regex  — 该做的事做了？
    { "type": "regex" , "pattern": "GET\\s+/rest/api/2/issue/[A-Z0-9-]+"      },

    absent_regex  ★ 已知坑没再踩？
    { "type": "absent_regex" , "pattern": "--format\\s+body"      }

  ]
}
```

✗ 无效演化 — 修了 fail, 但已 work 的也挂了

不变量轨道
已 work 的场景



退化!

修复目标轨道
旧 fail 场景



↓
改动在这里

✓ 有效演化 — fail 修了, 已 work 的不动

不变量轨道
已 work 的场景



保持

修复目标轨道
旧 fail 场景



↓
改动在这里

Runtime: AI Native

CLAUDE.md

Subagents

Hooks

Memory

Curation

Rules

Teams

Human: set goals → approve plans → review PRs → provide corrections

Skills Stack (3 Layers + Auth)

L2: Domain Workflow

diagnose-status

create-pr

debug-package

run-test

review-pr

write-procedure

...

L1.5: Domain Primitives

connect-db

run-sql

get-tenant

query-logs

check-validation

find-schema

...

L1: Foundation Skills

Jira

Slack

GitHub

Grafana

Outlook

Teams

Jenkins

Wiki

...

L0: Infrastructure

sig

gh

git

node

python

curl

...

~75%

自动化程度
从 ~50% 提升

50+

Skills
4 层架构

mins

Debug 时间
从半天 → 几分钟

领域知识编码在 **Skill** 里

人 + Agent team = 全栈团队输出

PART FOUR

愿景与展望

从人纠偏到自我进化

04



自动化程度演进

~50%

~75%

90%+

Before

手动 + 碎片化

Now

Skill 分层 + Forge

Future

自进化 Harness

未来：Skill 自动进化

自动检测退化

Self-Test 持续运行

自动修复 Skill

Agent Debugger + Evolve

跨任务迁移

结构化改进有复利



Key Takeaways

Agentic AICon

01

Auth 是 Agent 接入企业系统的第一道坎

零信任代理 · 无 token 上岸 · 浏览器接管

02

连接器只是起点 · 企业需要 Domain Skill 分层架构

30+ Skills · 复用 sigcli auth · 业务可组合

03

★ Skill 写完 ≠ 永远可用 — 需要演化闭环

trace → evidence → evolve → verify